

Universidad Tecnológica Nacional Facultad Regional San Rafael Ingeniería en Sistemas	UNIVERSIDAD TECNOLÓGICA NACIONAL	Desarrollo de Software
--	--	---------------------------

Trabajo Práctico: Test de Carga, Estrés y Optimización de Microservicio

Integrantes	Aaron Rebeco Bustos, Victoria Rios, Agustín Carvajal, Pablo de la Torre, Ana Gomez, Carolina Bravo
Repositorio	https://github.com/CNBRAVOCUENCA/extraccion-service
Fecha	01/10/2026

1. Objetivo

Diseñar, implementar y optimizar el microservicio de **extracción de texto y conversión de PDF a Markdown**, y evaluar su resiliencia, latencia y rendimiento bajo concurrencia extrema con **Grafana k6** (prueba spike, modelo cerrado) y **Vegeta** (carga fija, modelo abierto), comparándolo con el microservicio de referencia de la cátedra.

2. Resultados en una mirada

Prueba	Métrica	Profesor	Nuestro servicio
Vegeta 50 req/s · 30 s	Throughput efectivo	16,65 req/s	50,02 req/s
	Peticiones exitosas	998 / 1.500 (66,53 %)	1.500 / 1.500 (100 %)
	Timeouts (código 0)	501 (33,40 %)	0
	Latencia P50 / P95 / P99	14,89 s / — / —	6,4 ms / 16,5 ms / 30,6 ms
k6 spike 100 VUs · 40 s	Peticiones procesadas	1.037	14.818
	Throughput sostenido	25,35 req/s	370,4 req/s
	Tasa de error	0,00 %	0,00 %
	Latencia P50 / P90 / P95	1,88 / 7,83 / 8,80 s	0,086 / 0,56 / 0,76 s
	Latencia máxima	13,94 s	1,72 s

Ambas pruebas se ejecutaron con las mismas restricciones que exige el TP: 5 réplicas, 1,0 CPU y 1 GB de RAM por réplica.

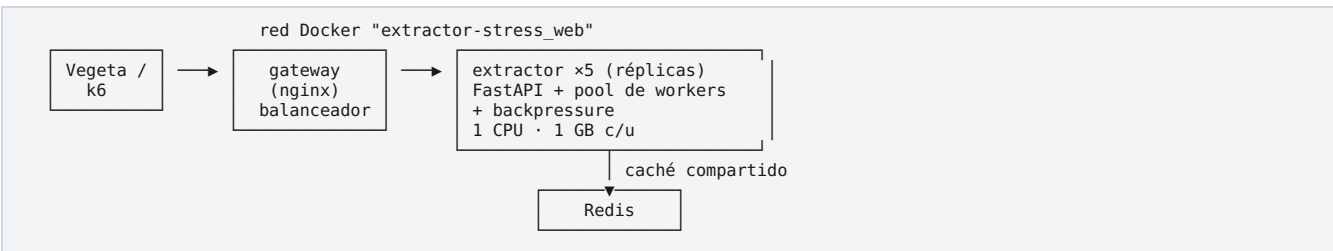
3. El endpoint

```
POST /extract
Entrada : PDF binario - multipart/form-data (campo "file") o el PDF crudo en el body
Salida : 200 application/json
{"content": "<texto en Markdown>", "page_count": N}
Errores : 400 sin archivo · 422 PDF inválido · 401 PDF con contraseña
503 servicio saturado (backpressure, con header Retry-After)
```

Es **síncrono**: el cliente envía el PDF y recibe el Markdown en la misma respuesta. No hay cola externa ni consulta posterior del resultado. La respuesta incluye el header X-Cache: HIT|MISS.

Conversión a Markdown. Con PyMuPDF se lee cada fragmento de texto con su tamaño de letra y estilo. El tamaño más frecuente se toma como cuerpo del texto; las líneas bastante más grandes se convierten en títulos (#, ##, ###), los fragmentos en negrita en ****texto****, las líneas con viñeta en ítems - y cada bloque del PDF en un párrafo. No usa librerías adicionales, así que el costo de CPU se mantiene bajo.

4. Arquitectura



Todo se levanta con un único comando: `docker compose up --build`.

Cada petición atraviesa tres controles dentro del extractor:

1. **Caché (Redis)**. La clave es el SHA-256 del PDF. Si ya fue procesado, el resultado se devuelve sin usar CPU. El caché es compartido por las 5 réplicas, así que la primera que procesa un PDF lo deja listo para las demás.
2. **Admisión / backpressure**. Un contador de peticiones en curso por réplica (`MAX_INFLIGHT`). Si se supera, se responde **503 inmediato** con `Retry-After`, en vez de encolar peticiones que terminarían en timeout.
3. **Pool de workers**. La extracción (CPU-bound) corre en un `ThreadPoolExecutor`, separada del event loop de FastAPI. El runtime HTTP queda libre para aceptar o rechazar rápido mientras los workers procesan.

Balanceador. nginx resuelve el nombre del servicio `extractor` con el DNS interno de Docker, que devuelve las IP de las 5 réplicas rotándolas. Además reintenta en otra réplica si una falla o responde 503 (`proxy_next_upstream`), y transmite el cuerpo sin bufferearlo entero (`proxy_request_buffering off`), lo que conviene con PDFs grandes.

Decisiones de diseño

Decisión	Motivo
FastAPI + Uvicorn	Async: separa la E/S (recibir el PDF, Redis) del trabajo de CPU, que va al pool.
PyMuPDF en vez de pypdf	Unas 19 veces más rápido (ver sección 5) y permite leer tamaños y estilos para el Markdown.
Backend intercambiable (<code>EXTRACTOR_BACKEND</code>)	Patrón Strategy / principio Abierto-Cerrado: se cambia por variable de entorno, sin tocar código. Permite medir el baseline.
1 worker por réplica	Con 1,0 CPU por réplica, más workers solo agregan cambios de contexto. El paralelismo viene de las réplicas.
nginx en vez de Traefik	Traefik necesita el socket de Docker y en Docker Desktop para Windows fallaba (error 400 de la API). nginx no depende del socket; el TP admite cualquier reverse proxy.
Estructuras livianas	El PDF se maneja como bytes una sola vez, en memoria (<code>BytesIO</code>), sin escribir a disco ni duplicar buffers. En caché se guarda solo el JSON de salida.
Twelve-Factor	Configuración por variables de entorno, procesos sin estado (el estado compartido está en Redis), logs a stdout, port binding, <code>restart: unless-stopped</code> .

5. Cuello de botella identificado

El límite es la **CPU de extracción**. Medido en un solo proceso, mediana de 5 corridas por PDF:

PDF	Tamaño	pypdf	PyMuPDF
01-liviano	170 KB	266 ms	16 ms
02-chico	290 KB	99 ms	35 ms
03-mediano	1,3 MB	87 ms	22 ms
04-grande	2,5 MB, 30 páginas	4.023 ms	162 ms
Promedio		1.119 ms	58 ms

Con pypdf, cada réplica (1 CPU) procesa menos de 1 PDF por segundo en promedio, y el PDF grande la bloquea 4 segundos. A 50 req/s en modelo abierto, la llegada supera ampliamente la capacidad: las peticiones se acumulan y expiran. Es el comportamiento que muestra el servicio de referencia (33 % de timeouts, P50 de 14,9 s).

Las optimizaciones atacan ese cuello de botella desde tres lados:

- **Backend más eficiente**: baja el costo por PDF unas 19 veces.
- **Caché compartido**: elimina el costo de CPU en los PDFs repetidos. Con 4 PDFs rotando, después de la primera extracción de cada uno casi todas las peticiones salen de caché.
- **Backpressure + réplicas**: 5 CPUs en paralelo, y si igual se satura, el excedente recibe 503 rápido en lugar de un timeout de 30 s.

6. Comparativa antes vs. después

Antes (servicio original): flujo asíncrono `POST /api/v1/extract` con `document_id`, que pedía el PDF a documentos-service; extracción con pypdf, un solo proceso, sin caché ni control de concurrencia, y salida en texto plano.

Después: `POST /extract` síncrono que recibe el PDF directo y devuelve Markdown, con PyMuPDF, 5 réplicas balanceadas, backpressure y caché Redis.

	Antes	Después
Costo de extracción promedio	1.119 ms	58 ms
PDF de 30 páginas	4.023 ms	162 ms
Salida	Texto plano	Markdown
Dependencias para extraer	documentos-service + MongoDB	Ninguna (PDF en el request)
Comportamiento ante saturación	Encola hasta el timeout	503 inmediato + reintento en otra réplica
Vegeta 50 req/s	—	100 % de éxito, P50 6,4 ms
k6 spike 100 VUs	—	14.818 peticiones, 0 % error

Para medir el sistema sin las optimizaciones con las mismas pruebas de carga, basta cambiar en `docker-compose.yml`:
`EXTRACTOR_BACKEND=pypdf`, `CACHE_ENABLED=false` y replicas: 1.

7. Proceso de investigación

Lo que se fue probando, en orden, y qué se aprendió en cada paso:

1. **Endpoint síncrono.** El servicio original era asíncrono y dependía de documentos-service. Se agregó `POST /extract` que recibe el PDF directo, manteniendo el endpoint anterior para la Saga del orquestador. Se desarrolló con TDD: primero los tests de la lógica (extracción, pool, caché, backpressure) y al final el endpoint.
2. **Separar HTTP de CPU.** Al ser CPU-bound, la extracción bloqueaba el event loop. Se movió a un `ThreadPoolExecutor` y se agregó un límite de peticiones en curso que responde 503. Se verificó con un test: con `MAX_INFLIGHT=2` y 6 peticiones simultáneas, 2 se procesan y 4 reciben 503.
3. **Elección del backend.** Se midieron pypdf y PyMuPDF con los mismos PDFs. La diferencia (unas 19 veces) definió el backend por defecto. Además, PyMuPDF da mejor texto (respeta los espacios entre palabras) y expone tamaños y estilos, necesarios para generar Markdown.
4. **Balanceador.** Primero se usó Traefik, que descubre las réplicas a través del socket de Docker. En Docker Desktop para Windows devolvía `API returned a 400 (Bad Request)` incluso fijando `DOCKER_API_VERSION`. Se reemplazó por nginx con resolución dinámica del nombre del servicio, que no necesita el socket.
5. **Primera corrida de Vegeta: 0 % de éxito** (750 respuestas 413 y 750 respuestas 502). Se reprodujo el entorno fuera de Docker (mismo `nginx.conf`, mismo extractor, mismos PDFs y Vegeta) y ahí funcionaba. La causa no estaba en el servicio sino en el camino de red: Vegeta apuntaba a `host.docker.internal`, que en Windows no llegaba al nginx del stack. Se resolvió ejecutando Vegeta y k6 **dentro de la red de Docker** contra `http://gateway/extract`, y se agregó un chequeo previo de una petición para detectar este tipo de problema antes de lanzar la carga.
6. **Primeros resultados (texto plano):** 100 % de éxito en Vegeta (P50 9,4 ms) y 8.088 peticiones sin errores en k6 (202 req/s).
7. **Markdown.** Se implementó la conversión con la información de tamaño y estilo de PyMuPDF, sin librerías nuevas, para no perder rendimiento. La clave del caché se versionó para no devolver resultados viejos en texto plano. Se repitieron ambas pruebas con el Markdown activo y los resultados mejoraron: Vegeta 100 % de éxito con P50 de 6,4 ms, y k6 14.818 peticiones (370 req/s) sin errores y P95 de 0,76 s. En ambas rondas el orden fue el mismo (Vegeta primero, k6 después), con el caché ya cargado.

8. Metodología: Code First + TDD

Se trabajó con enfoque **Code First**: primero la lógica de negocio con tests unitarios (TDD) y al final el endpoint. El contrato del endpoint es el JSON `{"content", "page_count"}` que pide el TP, y FastAPI genera el OpenAPI a partir del código. Los tests verifican ese contrato: multipart y body crudo devuelven 200 con el formato correcto, PDF inválido devuelve 422, servicio saturado devuelve 503 y la conversión genera títulos y negritas en Markdown.

9. Cómo reproducir

```
# 1. Levantar (5 réplicas + nginx + Redis)
docker compose up -d --build

# 2. Prueba de carga fija (Vegeta) y spike (k6)
.\tests\stress\vegeta\run-vegeta.ps1
.\tests\stress\k6\run-k6.ps1

# 3. Tests unitarios
pip install -e ".[dev,fast]"
pytest test/ -v
```

Los PDFs de prueba están en `tests/stress/pdfs`.